Welcome to CSC 340-02

Programming Methodology Week 3
Review Week 2
C++ Encapsulation, Polymorphism and Memory Management

Instructor: Joseph Hui
Email: jhui@sfsu.edu

C++ Syntax

1. Enables input and output ----------> `#include <iostream>`
2. Opens the standard namespace ----> `using namespace std;` // syntax structure allowing grouping of data & methods
3. Begins the function **main** ----------> `int main()` // main must be global and return int
4. A comment ----------> `// Computes the volume of a sphere of a given radius.`
5. Begins body of function ----------> `{`
6. Defines a constant ----------> `const double PI = 3.14159;` // constants do not change
7. Declares a variable ----------> `double radius;` // variables begin with a letter or underscore

8. Displays a prompt to the user -------> `cout << "Enter the radius of the sphere: ";`
9. Reads **radius** ----------> `cin >> radius;`

10. Declares and computes **volume** ---> `double volume = 4 * PI * radius * radius * radius / 3;`
11. Displays results ----------> `cout << "The volume of a sphere of radius "`
12. Statement continues ----------> `<< radius << " inches is " << volume`
13. Statement continues ----------> `<< " cubic inches.\n";`
14. Normal program termination -------> `return 0;`
15. Ends body of function ----------> `} // end program`

Multi-line comments start with /* and end with */

C++ Functions

- An argument passed by value is copied within the function.
- The function cannot alter the arguments passed in.

```
int computeMax(int x, int y)
{
   if (x > y)
      return x;
   else
      return y;
}  // end computeMax
```

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Functions

- An argument passed by reference is not copied but is accessed directly within the function.

```
void computeMax(int x, int y, int& larger)        // void function does not return a value
{
        larger = ((x > y) ? x : y):
}       // end computeMax

int main ()
{
         int a = 0;
         int b = 0;
         int largerAB = 0;
         cin >> a >> b;
         computeMax(a, b, largerAB);
         cout << "The larger of " << a << " and " << b << " is " << largerAB << ".\n";
}
```

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Passing Arrays to a Functions

- Arrays are always passed by reference to a function.
- Common ways to pass arrays to function:
  1. void modifyArray(int arr[]) // same as #2
  2. void modifyArray(int *arr) // same as #1
  3. void modifyPointer(int*& arr) // allows you modify array elements and pointer itself; useful when you want to change memory location

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Vectors

- A vector is similar to a one-dimensional array.
  - an object of a standard C++ class named vector
  - is a part of the Standard Template Library , or STL
    - The STL is a collection of data types you can use in your programs.
    - These data types are not part of the official C++ language, but they have been added to the built-in data types.
- To use a vector in your program, you must begin it with the following statements:

```
#include <vector>
using namespace std;
```

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Strings

- The C++ Standard Library provides the data type string. To use this library, you include the statements

```
#include <string>
using namespace std;
```

- You can concatenate two strings to form another string by using the + operator.
- You can manipulate a portion of a string by using the method

```
substr(position, length )
```

- The first argument specifies the position of the beginning of the substring (remember that 0 is the position of the first character in the string). The second argument is the length of the substring. For example, (e.g., title = "J Perfect's Diary");

```
title.substr(2, 7)
```

- is the string “ Perfect ”.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Important Themes in Programming

- Choose identifiers that **describe** their purpose—that is, that are self-documenting. Distinguish between keywords, such as int, and user-defined identifiers.
- Use a good **indentation** style to enhance the readability of a program. The layout of a program should make it easy for a reader to identify the program's modules.
- Use a **blank line** between method or function definitions.
- Within these definitions, you should **indent** individual blocks of code. These blocks are generally—but are not limited to—the actions performed within a control structure, such as a while loop or an if statement.
- Named constants make a program easier to modify, e.g.,

```
const int NUMBER_OF_MAJORS = 202;
```

- typedef statements make a program easier to modify; if your program performs floating point computations, and you need greater precision than float (change float to double)
  - typedef float RealType; // original statement
  - typedef long double RealType; // update to this statement in one spot

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Important Themes in Programming

- Two kinds of testing can be applied to a system:
  - Open-box testing (**white-box** or glass-box testing) - design tests knowing how the modules work.
    - choose a method's actual arguments in a way that exercises all lines of code in the method.
    - make sure that all branches of if and switch statements are executed.
  - Closed-box testing (**black-box** or functional testing) - you know the name of the method being tested, as well as its argument list, its return type, and its operation contract.
    - develop tests across a range of valid input, and then check the output against hand-calculated results.
    - Choose arguments outside of their valid range tests how the module handles incorrect input.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Points

- Header (.h) files in C++ provide a mechanism to partially separate the design of a class from the implementation in the source , or implementation (.cpp), file.
- The header file must also contain a description of the data fields for the class and any private methods used by the class
- Elements of the class
  - Private data fields // good practice
    - restricts access to the data field to just the class in which it is defined
    - Typically, provide methods—such as setItem (setters) and getItem (getters)— to access the data fields
    - Class controls how and whether other classes can access the data fields
  - If a derived class might need to access or modify the data fields of the base class, you should still make the data fields private but provide protected methods so that any derived classes can access or modify the data

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Inheritance

- Inheritance allows us to reuse and extend work we have already completed and tested.
- You can think of a base class as a parent class.
  - base class = superclass = parent class
- You can think of a derived class as a child class
  - derived class = subclass = child class
- Any instance of the derived class is also considered to be an instance of the base class and can be used in a program anywhere that an instance of the base class can be used
- When the keyword public is used with the base class, any of the publicly defined methods or data fields in the base class can be used by instances of the derived class.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ derived class

- A derived class
  - has the data fields and methods of the base class in addition to the members it defines,
  - it can access only publicly defined members from the base class
  - can also revise any inherited public method.
  - contains copies of inherited data fields,
  - does not contain a copy of the code for inherited methods
  - does not inherit constructors from parent class, but it can use it or other parent method to initialize the private data fields

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Overriding Base-Class Methods

- You can add as many new members to derived class as desired
- You can redefine inherited methods
  - Called overriding a base-class method.
- A method overrides a base-class method when
  - The two methods have the same name and parameter declarations
- Precede a method name with the base-class namespace indicator to tell the compiler to use the base-class version of the method
- Without a namespace indicator in front of the method name, the compiler assumes that the programmer is referring to the current class and inserts code to call that version of the method

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Virtual Methods, Abstract Classes

- Using keyword **virtual** in front of the prototype
  - Tells the C++ compiler that the code this method executes is determined at runtime
  - Allows an application using our class to take advantage of polymorphism when the methods are invoked
- Pure virtual method
  - Virtual method that has no implementation
- Abstract class
  - Has at least one pure virtual method

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Major Concepts for Week 3

- Encapsulation
  - Same concept as in Java
- Polymorphism
  - Key difference
- Memory Management
  - Many differences